



GRADIO

Lucas Rios

GRADIO



gradio

A interface deve ser tão simples quanto a função Python que ela chama

<https://www.gradio.app/>

GRADIO VS. STREAMLIT

Streamlit (Rerun Model): Toda vez que um usuário interage com um widget (move um slider, por exemplo), o script inteiro roda do topo até o final novamente. É imperativo e linear.

Gradio (Event-driven Model): Baseia-se em eventos. Se você clica em um botão, apenas a função vinculada a esse botão é executada. Isso permite criar interfaces muito mais complexas, com múltiplos fluxos de dados independentes, sem o custo computacional de recarregar o modelo ou os dados a cada clique.

GRADIO VS. STREAMLIT

Streamlit (Rerun Model): Toda vez que um usuário interage com um widget (move um slider, por exemplo), o script inteiro roda do topo até o final novamente. É imperativo e linear.

Gradio (Event-driven Model): Baseia-se em eventos. Se você clica em um botão, apenas a função vinculada a esse botão é executada. Isso permite criar interfaces muito mais complexas, com múltiplos fluxos de dados independentes, sem o custo computacional de recarregar o modelo ou os dados a cada clique.

GRADIO VS. STREAMLIT

- Maior quantidade de Componentes.
- Comunidade maior com mais projetos publicados (Hugging Face)
- Facilidade em aplicação de estilos (CSS)
- Facilidade de deploy e publicação direta, inclusive pelo Colab.

GRADIO VS. STREAMLIT

Streamlit (Rerun Model): Toda vez que um usuário interage com um widget (move um slider, por exemplo), o script inteiro roda do topo até o final novamente. É imperativo e linear.

Gradio (Event-driven Model): Baseia-se em eventos. Se você clica em um botão, apenas a função vinculada a esse botão é executada. Isso permite criar interfaces muito mais complexas, com múltiplos fluxos de dados independentes, sem o custo computacional de recarregar o modelo ou os dados a cada clique.

INTERFACE VS. BLOCKS

gr.Interface

Foi desenhada para a produtividade máxima. Ela exige três pilares:

fn: A função lógica (seu modelo).

inputs: O componente de entrada (pode ser uma string como "text" ou um objeto como `gr.Image()`).

outputs: Onde o resultado será exibido.

INTERFACE VS. BLOCKS

gr.Blocks

Construção de páginas e interfaces complexas

DEPLOY

```
demo.launch(pwa=True, share=True, inbrowser=True)
```

EXPLORANDO O SITE DO GRADIO

<https://www.gradio.app/>



[API](#)

[Guides](#)

[Themes](#)

[Custom Components](#)

[HTML Components](#)

[Community](#) ▼

HUGGING FACE

<https://huggingface.co/>

Criada em 2016, adquiriu o Gradio em 2021 e tem como objetivo democratizar o acesso ao aprendizado de máquina (Machine Learning) e à inteligência artificial (IA).

Permitindo que desenvolvedores compartilhem, explorem e utilizem projetos de IA, como modelos pré-treinados, datasets e aplicações.

Atualmente, a plataforma conta com mais de 850 mil modelos disponíveis, abrangendo tarefas como processamento de texto, áudio e imagem.

Uma das principais vantagens do Hugging Face é a disponibilização de modelos de código aberto, permitindo que desenvolvedores repliquem e adaptem esses modelos gratuitamente. Isso reduz custos e o impacto ambiental, já que o treinamento de modelos de grande porte consome muitos recursos.

